

Laporan Praktikum 4 Kontrol Cerdas

Nama : Levis Mega Nureska

NIM : 224308010

Kelas : TKA – 6A

Akun Github (Tautan) : <https://github.com/levismeganureska>

Student Lab Assistant : Muhammad Fatih Anfa Adhima

1. Judul Percobaan

Judul : Implementasi Reinforcement Learning untuk Pengendalian Otonom dalam Lingkungan Simulasi

2. Tujuan Percobaan

Pada praktikum ini, mahasiswa akan dapat:

- Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
- Mengimplementasikan agen RL menggunakan algoritma Deep Q-Network (DQN).
- Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
- Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
- Menggunakan GitHub untuk version control dan dokumentasi praktikum.

3. Landasan Teori

Reinforcement Learning adalah proses mapping (pemetaan) dari situasi yang ada di environment (states) ke bentuk aksi (behavior) agar dapat memaksimalkan reward. Agent yang bertindak sebagai sang learner tidak perlu diberitahukan behavior apakah yang akan sepatutnya dilakukan, atau dengan kata lain, sang learner belajar sendiri dari pengalamannya. Salah satu konsep utama dalam Reinforcement Learning yaitu Agent. Dalam Reinforcement Learning (RL), Agent adalah entitas atau program yang mengambil keputusan dan berinteraksi dengan lingkungan untuk mencapai tujuan tertentu. Agent mengambil tindakan, mendapatkan feedback, dan belajar dari pengalaman untuk meningkatkan performanya. Tugas utama dari Agent yaitu mengamati lingkungan, memilih tindakan atau action, menerima feedback, dan memperbarui strategi. Maka dari itu Agent adalah inti dari Reinforcement Learning yang bertindak berdasarkan pengamatan lingkungan, menerima reward, dan terus belajar untuk meningkatkan kemampuannya.

Deep Q-Network (DQN) adalah algoritma dalam reinforcement learning yang menggabungkan Q-learning dengan jaringan saraf dalam untuk menyelesaikan masalah kompleks, sehingga memungkinkan agen untuk belajar dari pengalaman dan membuat keputusan dalam lingkungan yang rumit. Algoritma ini sangat berguna untuk menangani masalah dengan ruang keadaan yang besar, seperti pengenalan gambar dan kontrol robot, dengan memanfaatkan kemampuan jaringan saraf untuk mengekstrak fitur dari data secara efisien dan efektif. DQN bekerja dengan cara memanfaatkan jaringan saraf untuk memperkirakan nilai Q, yang merupakan nilai dari setiap aksi yang mungkin diambil dalam

suatu keadaan tertentu. Dengan menggunakan pemutaran ulang pengalaman, DQN menyimpan pengalaman masa lalu dalam memori dan mengambil sampel acak dari pengalaman tersebut untuk memperbarui model, sehingga meningkatkan stabilitas dan efisiensi pembelajaran. Selain itu, DQN juga menerapkan strategi eksplorasi epsilon-greedy, yang memungkinkan agen untuk menyeimbangkan antara eksplorasi tindakan baru dan eksploitasi tindakan yang sudah diketahui, sehingga agen dapat menemukan strategi yang optimal seiring waktu.

4. Analisis dan Diskusi

Analisis :

1. Bagaimana Performa Agen dalam Mengontrol Environment CartPole?

Agen menggunakan **Deep Q-Network (DQN)** untuk belajar mengontrol lingkungan **CartPole**. Selama pelatihan, agen pada awalnya mengambil tindakan secara acak karena nilai **epsilon** yang tinggi (**1.0**), tetapi seiring waktu, ia mulai belajar dari pengalaman yang tersimpan dalam **replay buffer**. Model diperbarui menggunakan **Q-learning**, dan eksplorasi berkurang secara bertahap dengan menurunnya nilai **epsilon** (**epsilon decay**). Jika hanya menggunakan **30 episode**, agen belum memiliki cukup pengalaman untuk membuat keputusan yang optimal, sehingga performanya masih rendah dan cenderung acak. Namun, dengan **1000 episode**, agen memiliki lebih banyak kesempatan untuk mengeksplorasi berbagai strategi, sehingga dapat mengoptimalkan kebijakan pengambilan keputusan dan mencapai skor yang lebih tinggi secara konsisten. Dengan jumlah episode yang lebih banyak, model menjadi lebih stabil, epsilon mendekati **0.01**, dan agen lebih sering memilih tindakan terbaik berdasarkan pengalaman yang telah dipelajari.

2. Bagaimana Perubahan Parameter Mempengaruhi Kinerja Agen?

Beberapa parameter penting yang mempengaruhi kinerja agen meliputi:

- Gamma (0.95) → Faktor diskonto, semakin tinggi nilainya, semakin fokus agen pada reward jangka panjang.
- Epsilon (1.0 → 0.01, decay 0.995) → Laju eksplorasi, jika terlalu cepat turun, agen bisa terjebak dalam lokal optima sebelum cukup mengeksplorasi tindakan yang lebih baik.
- Learning rate (0.001) → Jika terlalu besar, model bisa sulit untuk konvergen, sementara jika terlalu kecil, model belajar terlalu lambat.
- Ukuran hidden layer (24, 24 neurons) → Struktur jaringan berpengaruh terhadap kemampuan agen dalam mengekstraksi pola dari data.
- Batch size (64) → Jika terlalu kecil, pembelajaran bisa terlalu bervariasi (noisy); jika terlalu besar, pembelajaran bisa lambat.

3. Apa Tantangan yang Muncul Selama Pelatihan Agen RL?

Beberapa tantangan yang sering dihadapi selama pelatihan agen RL pada environment CartPole meliputi:

- Ketidakstabilan dalam Pembelajaran : Jika learning rate terlalu besar atau parameter tidak diatur dengan baik, agen bisa gagal mencapai konvergensi.
- Eksplorasi vs. Eksploitasi : Jika **ε-greedy policy** tidak dikonfigurasi dengan baik, agen bisa terjebak dalam strategi suboptimal karena eksploitasi terlalu dini atau eksplorasi yang tidak cukup.
- Overfitting ke Lingkungan Latihan : Agen yang terlalu banyak berlatih dalam satu environment mungkin kesulitan beradaptasi jika environment sedikit berubah.
- Efisiensi Sampel : **DQN** membutuhkan banyak episode untuk mencapai performa optimal, yang memakan banyak waktu dan sumber daya komputasi.

Diskusi :

1. Apa Perbedaan Utama antara Reinforcement Learning dan Metode Supervised Learning dalam Sistem Kendali?

Supervised Learning

- Memerlukan dataset berlabel yang telah tersedia sebelumnya.
- Model dilatih dengan mencocokkan input dengan output berdasarkan contoh yang diberikan.
- Digunakan untuk tugas prediktif seperti klasifikasi dan regresi.
- Contoh dalam sistem kendali: Memprediksi suhu menggunakan data sensor pada sistem HVAC.

Reinforcement Learning

- Tidak membutuhkan data berlabel, melainkan belajar melalui interaksi dengan lingkungan.
- Agen memperoleh reward sebagai umpan balik untuk menentukan strategi optimal.
- Cocok untuk pengambilan keputusan dalam lingkungan yang dinamis dan terus berubah.
- Contoh dalam sistem kendali: Mengendalikan robot agar tetap seimbang saat berjalan dengan belajar dari pengalaman.

2. Bagaimana Strategi Eksplorasi (Exploration) dan Eksploitasi (Exploitation) Dapat Dioptimalkan pada Agen RL?

Beberapa Metode untuk Mengoptimalkan Strategi Eksplorasi dan Eksploitasi

1. **Upper Confidence Bound (UCB)** – Memilih tindakan dengan mempertimbangkan keseimbangan antara eksplorasi dan perkiraan reward yang diharapkan.
2. **Boltzmann Exploration** – Mengurangi eksplorasi yang sepenuhnya acak dengan lebih memprioritaskan tindakan yang memiliki peluang reward lebih tinggi.
3. **Intrinsic Motivation** – Memberikan reward tambahan kepada agen untuk mendorong eksplorasi hal baru, mirip dengan bagaimana manusia belajar melalui rasa ingin tahu.

3. Potensi Aplikasi RL dalam Sistem Kendali Nyata yang Dapat Diimplementasikan

Reinforcement Learning memiliki banyak aplikasi dalam sistem kendali di dunia nyata, termasuk:

1. Kendaraan Otonom
 - RL digunakan dalam self-driving cars untuk pengambilan keputusan seperti navigasi, penghindaran rintangan, dan manuver optimal.
2. Robotika Industri
 - Digunakan dalam pengendalian lengan robot untuk tugas seperti perakitan otomatis dan manipulasi objek dengan presisi tinggi.

5. Assignment

Pada praktikum ini, algoritma Deep Q-Network (DQN) digunakan untuk melatih agen dalam lingkungan MountainCar-v0. Tujuan utama agen adalah mencapai puncak bukit dengan memilih aksi yang paling optimal. Implementasi dimulai dengan mengimpor pustaka seperti Gymnasium untuk pengelolaan lingkungan, NumPy untuk operasi numerik, random untuk eksplorasi aksi, serta deque dari collections untuk menyimpan pengalaman agen. Selain itu, TensorFlow.Keras digunakan dalam membangun dan melatih model jaringan saraf.

Agen DQN didefinisikan dalam kelas DQNAgent, yang diinisialisasi berdasarkan ukuran keadaan dan jumlah aksi yang tersedia. Beberapa parameter utama dalam algoritma ini meliputi gamma sebagai faktor diskon reward masa depan, epsilon untuk eksplorasi, serta learning rate yang menentukan kecepatan optimasi model.

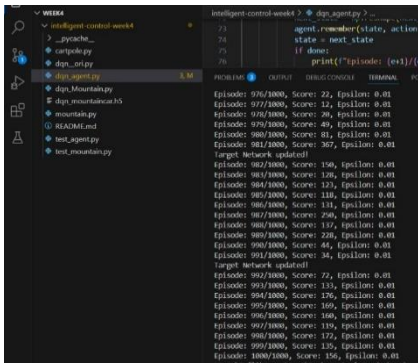
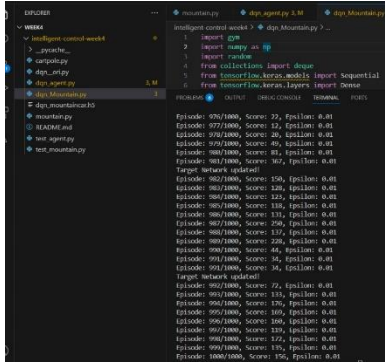
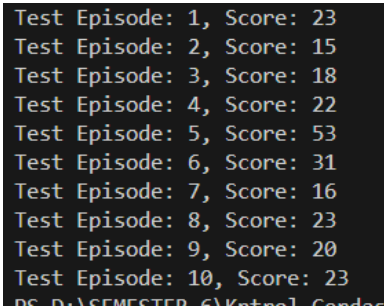
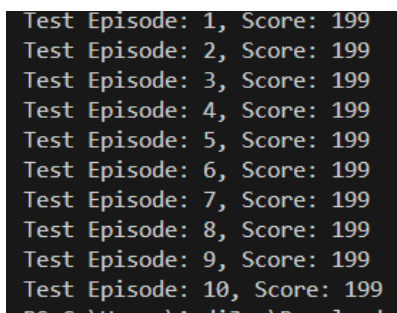
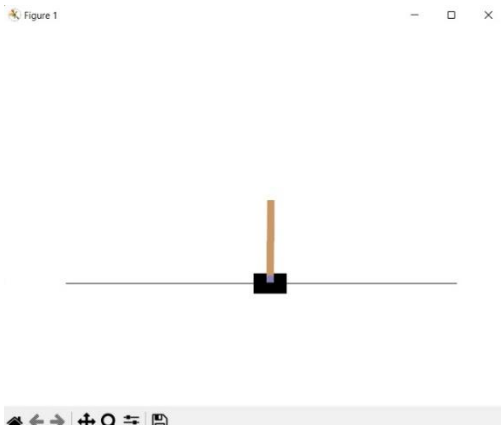
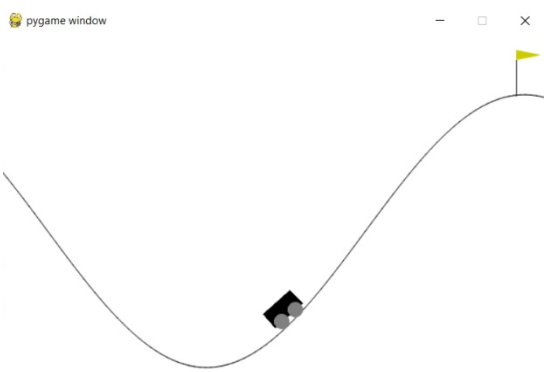
Dalam eksperimen CartPole-v1, agen bertugas menjaga keseimbangan tiang di atas gerobak. Karena lingkungan ini lebih sederhana dan dapat diprediksi, agen bisa belajar dengan cepat.

Setiap kali berhasil menjaga keseimbangan, agen mendapatkan reward positif. Pelatihan biasanya selesai dalam beberapa ratus episode karena agen relatif mudah mencapai performa optimal.

Sebaliknya, MountainCar-v0 memiliki tantangan lebih kompleks. Agen harus membangun momentum dengan bergerak maju dan mundur sebelum dapat mencapai puncak bukit karena tenaga mobil tidak cukup untuk langsung sampai tujuan. Reward lebih sulit diperoleh, dan agen sering gagal di awal pembelajaran. Karena tingkat kesulitan yang lebih tinggi, MountainCar-v0 memerlukan model yang lebih kompleks atau teknik tambahan untuk meningkatkan stabilitas pelatihan. Proses belajar dalam lingkungan ini membutuhkan lebih banyak episode, dengan nilai reward yang fluktuatif, sehingga diperlukan penyesuaian agar agen bisa mencapai performa optimal secara konsisten.

6. Data dan Output Hasil Pengamatan

Sajikan data dan hasil yang diperoleh selama percobaan. Gunakan tabel untuk menyajikan data jika diperlukan.

No	Variabel	Hasil Pengamatan	
		Cartpole	Mountain car
1	Hasil Score (DQN)		
2	Hasil Score (Test)		
3	Hasil Run		

7. Kesimpulan

CartPole-v1:

- Berfokus pada menjaga stabilitas dan keseimbangan tongkat.
- Reward diberikan setiap kali agen berhasil mempertahankan keseimbangan.
- Pengujian dilakukan dengan nilai epsilon rendah untuk memastikan model yang telah dilatih dapat dieksploitasi secara optimal.
- Visualisasi kinerja agen ditampilkan melalui animasi.

MountainCar-v0:

- Memiliki tingkat kesulitan lebih tinggi karena reward standar hanya diperoleh ketika mobil mencapai puncak bukit.
- Algoritma ditingkatkan dengan modifikasi reward agar agen lebih cepat mencapai tujuan.
- Menggunakan replay memory serta penurunan epsilon secara bertahap untuk meningkatkan efektivitas eksplorasi.
- Model dilatih dalam beberapa episode sebelum diuji.

8. Saran

CartPole:

- Stabilitas pembelajaran dapat ditingkatkan dengan menerapkan metode Dueling DQN atau Double DQN.
- Efisiensi dapat diperbaiki dengan penggunaan frame skipping atau penerapan reward shaping.
- Eksplorasi suboptimal dapat diatasi dengan mengimplementasikan Curiosity-driven Exploration.

MountainCar:

- Penggunaan prioritized experience replay membantu agen lebih cepat belajar dari pengalaman yang paling berpengaruh.
- Efisiensi dalam lingkungan dengan reward yang jarang dapat ditingkatkan menggunakan metode actor-critic seperti A2C atau DDPG.
- Transfer learning dari model yang telah dilatih di lingkungan serupa dapat mempercepat proses konvergensi.

9. Daftar Pustaka

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction (2nd Edition)*. MIT Press.
- Silver, D. (2015). *Lecture Notes on Reinforcement Learning*. University College London (UCL).
- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). *Human-level control through deep reinforcement learning*. *Nature*, 518(7540), 529–533.